

Section 6.1: System Consistency Models & Partition Theorems

Part of a 3-File Distributed Systems Series (File 1 of 3)

Study Note: This section is heavily math-oriented. Extra weight is given to worked numeric examples and proof-by-contradiction style correctness arguments throughout. For every theorem, an impossibility argument is included alongside the practical implication.

1. CAP Theorem — Deep Dive

The Formal Statement

CAP states: *A distributed system cannot simultaneously guarantee all three of the following:*

Property	Meaning
Consistency	Every read receives the most recent write or an error
Availability	Every request receives a (non-error) response
Partition Tolerance	The system continues operating despite arbitrary message loss

Key insight most candidates miss: Partition tolerance is not optional in a real network. Networks drop packets. Therefore the real choice is: **during a partition, do you sacrifice C or A?**

Proof by Contradiction: Why You Cannot Have All Three

Assume a system S claims to be CAP-complete (consistent, available, and partition-tolerant).

Setup: - Two nodes: N1 and N2 - Client writes value `v=1` to N1 - Network partition occurs — N1 and N2 cannot communicate - Client reads from N2

The contradiction: - For **Availability**: N2 must respond (not error) - For **Partition Tolerance**: N2 cannot contact N1 to get the latest write - For **Consistency**: N2 must return `v=1` (the latest write)

But N2 *cannot* return `v=1` without contacting N1, and it *cannot* contact N1 (partition). Therefore it either: - Returns stale data → violates Consistency - Returns an error → violates Availability

Conclusion: CAP-completeness is impossible. QED.

CP vs. AP: When to Choose Which

Dimension	CP Systems	AP Systems
Examples	HBase, MongoDB (strong mode), Redis Cluster	Cassandra, DynamoDB, CouchDB
During partition	Reject writes; return errors	Accept writes; serve stale reads
After partition heals	No reconciliation needed (no divergence)	Must merge conflicts
Use when	Correctness is non-negotiable (finance, inventory)	Availability is non-negotiable (social feeds, analytics)
Failure mode	Timeout / 503 errors during split	Users see inconsistent state

2. PACELC Theorem — The Full Picture

CAP only describes behavior *during* a partition. PACELC adds the normal-operation dimension.

```
If Partition:    Availability vs. Consistency  (CAP)
Else (normal):  Latency      vs. Consistency  (new)
```

Worked Numeric Example: Latency Cost of Consistency

Scenario: 5-replica system, 3 regions (US, EU, APAC).

Eventual Consistency (low latency path):

Write hits US node → ACK immediately → 2ms response
Background replication: US→EU ~80ms, US→APAC ~160ms
User in APAC reads within 160ms window → sees stale data

Strong Consistency (high latency path):

Write hits US node → waits for quorum (3 of 5 ACKs)
Best case: 2 ACKs within US in ~5ms
Third ACK from EU: +80ms round trip
Total write latency: ~85ms
APAC reads always see latest → no stale data

The PACELC trade-off in numbers: Strong consistency costs ~83ms extra per write to guarantee freshness. For 10,000 writes/sec, this is $10,000 \times 83\text{ms}$ of accumulated latency overhead vs. eventual consistency.

Real-World PACELC Classification

System	Partition	Normal	Notes
Spanner	CP	LC (High latency)	TrueTime ensures bounded uncertainty
DynamoDB	AP	EL (Low latency)	Tunable to strong consistency per-request
Cassandra	AP	EL (Low latency)	Tunable quorum
Zookeeper	CP	LC	Optimized for reads, slow writes
CockroachDB	CP	LC	Serializable isolation across regions

3. Distributed Consistency Spectrum

Think of consistency models as a spectrum from strongest (most correct, most expensive) to weakest (most scalable, most stale-prone).

STRONGEST ————— WEAKEST
Linearizable → Sequential → Causal → Read-Your-Writes → Eventual

3.1 Linearizability (Strong Consistency)

Formal property: Every operation appears to take effect atomically at some point between its invocation and its response. All clients observe a single consistent timeline.

Proof by contradiction (why it's hard):

Suppose two clients C1 and C2 both read a register, and then both write to it. Linearizability requires:
- A single global timeline - Real-time ordering preserved

In a distributed system with propagation delay d , if C1 completes a write at time T , C2 must not observe the old value after time $T + d$. But C2 cannot know what T is without global clock synchronization. Google Spanner solves this with TrueTime, which bounds clock uncertainty to $\epsilon \approx 7\text{ms}$, allowing safe commit windows.

Implementation cost:

```
Raft consensus for linearizable writes:  
Leader receives write  
→ Appends to log  
→ Replicates to majority ( $N/2 + 1$  nodes)  
→ Majority ACK  
→ Commits + responds to client  
  
Minimum latency = 1 network round trip to majority  
For 5-node cluster across 3 data centers: ~50-200ms
```

When to use: Financial ledgers, inventory counters, leader election, distributed locks.

3.2 Sequential Consistency

Property: All processes agree on the same order of operations, and that order respects each process's own program order — but does NOT need to respect real-time ordering.

Key difference from linearizability: - Linearizable: if A completes *before* B starts in real time, A precedes B globally - Sequential: A precedes B in program order *within one process*, but real-time between processes is not enforced

Practical implication: Can have “stale reads” from the perspective of wall clock time, but within one client session, ordering is consistent.

3.3 Causal Consistency

Property: If write A *causally precedes* write B (A happened-before B), then every process observes A before B. Concurrent writes (no causal link) can be observed in any order.

Vector Clock Example (worked):

Three nodes: N1, N2, N3. Clocks initialized [0,0,0].

Step 1: N1 writes X=1 → N1 clock becomes [1,0,0]

Step 2: N1 → N2 message → N2 clock merges: [1,1,0]

Step 3: N2 writes Y=2 → N2 clock becomes [1,2,0]

Step 4: N3 writes Z=3 → N3 clock: [0,0,1] (no message from N1/N2 yet)

Now: Is Y=2 causally related to Z=3?

N2 clock = [1,2,0], N3 clock = [0,0,1]

Neither dominates the other → CONCURRENT

Can be seen in any order.

Is Y=2 causally related to X=1?

N1 clock [1,0,0] < N2 clock [1,2,0] component-wise?

Yes: every component of [1,0,0] ≤ [1,2,0] and at least one is strictly less.

→ X=1 causally precedes Y=2. Must be observed in that order.

When to use: Social media posts and replies (a reply must appear after the post it's replying to), collaborative document edits.

3.4 Read-Your-Own-Writes (Session Guarantee)

Property: Within a single client session, after a write completes, all subsequent reads by that same client return that write or a later one.

Implementation options:

Option 1: Sticky sessions

Route client C always to replica R

All writes and reads hit R

Simple but: R failure breaks session

Option 2: Version tokens

Write to primary → get version token v=42

Read from any replica with header: "min-version: 42"

Replica serves request only if caught up to v≥42

Otherwise: wait or redirect to primary

When to use: User profile updates (user sets avatar, immediately sees it), e-commerce cart (add item, immediately see it in cart).

3.5 Eventual Consistency

Property: If no new writes occur, all replicas will converge to the same value eventually. No timing guarantee.

Anti-Entropy Mechanism (Merkle Trees — worked example):

Replica A and Replica B have diverged.

Step 1: Build Merkle tree for each replica's key space

Replica A:	Replica B:
Root: hash(AB)	Root: hash(AB')
├ Left: hash(keys 1-500)	├ Left: hash(keys 1-500) [SAME]
└ Right: hash(keys 501+)	└ Right: hash(keys 501+) [DIFFERENT]

Step 2: Compare roots → differ → go deeper

Step 3: Left subtrees match → skip

Step 4: Right subtrees differ → sync only keys 501-1000

Result: $O(\log N)$ comparison to find divergence, sync only divergent subset.

Without Merkle trees: $O(N)$ full comparison every time.

4. Strong vs. Eventual Consistency — When and Where

Decision Framework

Ask these questions in order:

1. Is incorrect data worse than no data?
YES → CP (strong consistency)
NO → AP (eventual consistency)
2. Do concurrent writes need conflict resolution?
YES → Need CRDTs or vector clocks
NO → Timestamps sufficient (last-write-wins)
3. What is your latency budget?
<10ms → Strong consistency is likely impossible cross-region
>100ms acceptable → Can afford quorum reads/writes
4. What is your failure blast radius?
User sees wrong balance → Strong consistency required
User sees 1000 vs 1001 likes → Eventual acceptable

Comparison Table: Business Domain → Consistency Model

Domain	Model	Reason
Bank transfers	Linearizable	Double-spend prevention
Flight seat booking	Linearizable	Overbooking prevention
Social media likes	Eventual	Counter accuracy not critical
Shopping cart	Read-your-writes	User expectation only
DNS records	Eventual	Propagation delay acceptable
Distributed lock	Linearizable	Correctness required
Leaderboard rankings	Causal	Relative order matters
User feed	Causal	Post before reply

5. ACID vs. BASE — Data Architecture Archetypes

ACID (Strong Guarantees)

Property	Definition	Implementation
Atomic	Transaction is all-or-nothing	Write-ahead log + rollback
Consistent	DB moves from valid state to valid state	Constraint checking
Isolated	Concurrent transactions don't interfere	MVCC / locking
Durable	Committed data survives failures	fsync, replication

Isolation levels (weakest to strongest):

Read Uncommitted → Read Committed → Repeatable Read → Serializable
(dirty reads ok) (no dirty reads) (no non-repeatable) (no phantoms)

Numeric example — Isolation anomaly:

Serializable isolation prevents phantom reads:

```
T1: SELECT COUNT(*) FROM orders WHERE status='pending' → 5
T2: INSERT INTO orders (status) VALUES ('pending') → commits
T1: SELECT COUNT(*) FROM orders WHERE status='pending' → 6 (phantom!)
```

Under Serializable: T1's second read still returns 5, or T1 is aborted.

BASE (Availability Guarantees)

Property	Definition
Basically Available	System responds even if data is stale/inconsistent
Soft State	State may change over time without input
Eventually Consistent	Will converge given time

Hybrid: Saga Pattern for Long-Running Transactions

Problem: ACID across microservices requires distributed locks → deadlocks at scale.

Solution: Saga = sequence of local ACID transactions + compensating transactions.

Order Saga:

```
T1: Create order (Orders DB) – local ACID
T2: Reserve inventory (Inventory DB) – local ACID
T3: Charge payment (Payments DB) – local ACID
T4: Confirm order (Orders DB) – local ACID
```

If T3 fails:

```
C2: Release inventory (compensate T2)
C1: Cancel order (compensate T1)
```

Key property: No global lock. Each step is independently committed.

Risk: Between T2 commit and C2 compensate, inventory is temporarily inconsistent.

Mitigation: Design compensating transactions to always succeed (idempotent + retry).

6. Interview Problems with Full Solutions

Problem 1: Read-Your-Writes for Social Media Posts

Scenario: A social media platform uses Cassandra (eventual consistency by default, multi-region). Users complain they post something and then don't see it in their feed.

Root cause analysis:

```
User writes post → US-East primary (confirmed)
User refreshes feed → load balancer routes to US-West replica
US-West replication lag: ~500ms
User sees old feed (their post is missing)
```

Solution — Consistency Zones with Version Tokens:

```
class PostService:
    def create_post(self, user_id: str, content: str) -> Post:
        post = Post(id=uuid7(), user_id=user_id, content=content)

        # Write to primary with LOCAL_QUORUM (within region)
        session.execute(
            "INSERT INTO posts (id, user_id, content, ts) VALUES (?, ?, ?, ?)",
            [post.id, user_id, content, time.time()],
            consistency_level=ConsistencyLevel.LOCAL_QUORUM
        )

        # Return version token = logical write timestamp
        return post, {"min_ts": post.ts}

    def get_feed(self, user_id: str, session_token: dict) -> List[Post]:
        min_ts = session_token.get("min_ts", 0)

        # Read with LOCAL_QUORUM to guarantee seeing own writes
        posts = session.execute(
            "SELECT * FROM posts WHERE user_id=? LIMIT 20",
            [user_id],
            consistency_level=ConsistencyLevel.LOCAL_QUORUM
        )

        # If own posts not yet visible (cross-region scenario), read from primary
        own_posts = [p for p in posts if p.user_id == user_id and p.ts >= min_ts]
        if not own_posts and min_ts > 0:
            # Fallback: direct read from primary region
            return self.read_from_primary(user_id)

        return posts
```

Design decision: LOCAL_QUORUM (not QUORUM) avoids cross-region latency while still guaranteeing intra-region read-your-writes. Cross-region consistency of others' posts remains eventual — which is acceptable.

Problem 2: Designing Consistency Levels for a Banking API

Scenario: Design the consistency model for a fintech API supporting: balance reads, transfers, and transaction history.

Analysis per operation:

Operation	Model Required	Reason
Read balance	Linearizable	User may have just transferred — must see latest
Initiate transfer	Linearizable	Prevents double-spend
Read transaction history	Sequential	Order matters; slight staleness ok
Analytics aggregates	Eventual	Approximate totals sufficient

Implementation:

```
class BankingAPI:
    def get_balance(self, account_id: str) -> Decimal:
        # Linearizable read via Raft leader
        # ALL replicas must be checked; leader quorum read
        return db.execute(
            "SELECT balance FROM accounts WHERE id = ?",
            consistency=LINEARIZABLE, # routes to Raft leader
        )

    def transfer(self, from_id: str, to_id: str, amount: Decimal):
        # Serializable transaction — uses distributed lock via Raft
        with distributed_lock([from_id, to_id], timeout=5):
            from_balance = self.get_balance(from_id)
            if from_balance < amount:
                raise InsufficientFundsError()

            db.execute("UPDATE accounts SET balance = balance - ? WHERE id = ?",
                       [amount, from_id], consistency=LINEARIZABLE)
            db.execute("UPDATE accounts SET balance = balance + ? WHERE id = ?",
                       [amount, to_id], consistency=LINEARIZABLE)

    def get_history(self, account_id: str) -> List[Transaction]:
        # Sequential consistency — slightly stale OK
```

```

        return db.execute(
            "SELECT * FROM transactions WHERE account_id = ? ORDER BY ts DESC
LIMIT 100",
            consistency=SEQUENTIAL
        )

```

Trade-off articulation for interview: The transfer operation acquires distributed locks on both account IDs sorted lexicographically (prevents deadlock). The 5-second timeout prevents lock starvation. Linearizable reads ensure no phantom balance exists between the read and write within the lock boundary.

Problem 3: Conflict Resolution with CRDTs for Offline-First App

Scenario: A note-taking app allows offline edits. When two devices sync, there can be conflicting edits to the same note. How do you handle this without losing data?

Option A: Last-Write-Wins (LWW)

```

Device A edits note at T=100: "Hello World"
Device B edits note at T=101: "Hi There"
Sync: T=101 wins → "Hi There" (Device A's edit silently lost)

```

```

Problem: Data loss. A's edit is gone.
When to use: Acceptable only for truly independent fields (avatar URL, status
flag).

```

Option B: Operational Transform (used by Google Docs)

```

Complex, requires central server to order operations.
Not suitable for truly peer-to-peer or offline-first.

```

Option C: CRDT (Conflict-free Replicated Data Type) — Recommended

```

# G-Counter CRDT example (grow-only counter, e.g., likes)
class GCounter:
    def __init__(self, node_id: str):
        self.node_id = node_id
        self.counts = {} # {node_id: count}

    def increment(self):
        self.counts[self.node_id] = self.counts.get(self.node_id, 0) + 1

    def value(self) -> int:
        return sum(self.counts.values())

    def merge(self, other: 'GCounter') -> 'GCounter':

```

```

# Take max of each node's count – always safe, no conflicts
merged = GCounter(self.node_id)
all_nodes = set(self.counts) | set(other.counts)
for node in all_nodes:
    merged.counts[node] = max(
        self.counts.get(node, 0),
        other.counts.get(node, 0)
    )
return merged

# Numeric example:
# Device A: {A: 3, B: 0} – incremented 3 times offline
# Device B: {A: 0, B: 5} – incremented 5 times offline
# Merge:    {A: 3, B: 5} → value = 8 (correct: 3+5)
# No conflict, no data loss, mathematically provable convergence.

```

Why CRDTs are correct (proof sketch): CRDT merge is commutative ($A \text{ merge } B = B \text{ merge } A$), associative ($(A \text{ merge } B) \text{ merge } C = A \text{ merge } (B \text{ merge } C)$), and idempotent ($A \text{ merge } A = A$). These three properties guarantee that regardless of merge order or duplicated messages, all replicas converge to the same value. This is a mathematical proof of correctness, not just engineering convention.

When to use CRDT vs. strong consistency: - CRDT: Collaborative editing, shopping cart (add-only), like counts, presence indicators - Strong consistency: Seat reservations, financial transfers, any “reserve exactly one” scenario

Quick Reference: Consistency Model Cheat Sheet

```

Linearizable: All ops ordered globally, real-time respected. Slowest. Most correct.
Sequential:  All ops ordered globally, program order respected. Slightly faster.
Causal:      Only causally related ops ordered. Good for social/collab apps.
Read-My-Writes: Session guarantee only. Simple, cheap. Good for UX consistency.
Eventual:    All replicas converge eventually. Fastest. Accept temporary divergence.

Rule of thumb:
Money moves      → Linearizable
Social features  → Causal or Eventual
User session     → Read-My-Writes
Analytics        → Eventual

```